Sistemas operativos

Jaime Nazar Anchorena

Contents

Practica 1: POSIX **2**
- Entorno de desarrollo . . . 2
- Crear procesos . . . 2
- `ps` . . . 3
- Copy on write . . . 3
- File management . . . 3
- `pipe` . . . 5
  - dup . . . 5

Teorica 1: Introduccion **5**
- Sistemas operativos . . . 5
  - Administrador de recursos . . . 6
- Revision hardware . . . 6
  - Procesador . . . 6
  - Memoria . . . 7
  - Dispositivos I/O . . . 7
  - Booteo . . . 8
- Tipos de SO . . . 8
- System calls . . . 8
  - Read . . . 9
  - Waitpid . . . 9
  - Kill . . . 9
  - Fork . . . 9
- A donde va la salida estandar . . . 9
- Estructura de un sistema operativo . . . 10
  - Monolithic systems . . . 10
  - Microkernels . . . 10
  - Virtual machines . . . 11

Practica 1: POSIX

Entorno de desarrollo

Trucos de bash:

- `ctrl + r` para buscar historial
- `!!` expande con el ultimo comando (`$sudo !!`)
- `file.{jpg, png}` expande a `file.jpg file.png`
- `$(...)` da el output de un comando (`touch file-$(date -I)`)
- `ctrl + a` para ir al comienzo de la linea
- `ctrl + e` para ir al final de la linea
- `cd -` vuelve al dir que estabas antes
- `fc` fix command, abre el ultimo comando en un editor y corre la version editada
- `type` indica de si algo es builtin, program, o alias (`type ping`)

En bin/ estan todos los comandos de administracion, para configurar el sistema. Por otro lado, en bin/ estan los comandos de uso general.

Crear procesos

Se pueden crear utilizando `fork`, syscall que la llamo una vez y vuelve dos veces, el proceso original y el que se creo.

Los dos procesos son identicos salvo el valor de retorno de `fork`. Si `fork` no puede crear un proceso(raro, pero puede pasar), devuelve -1.

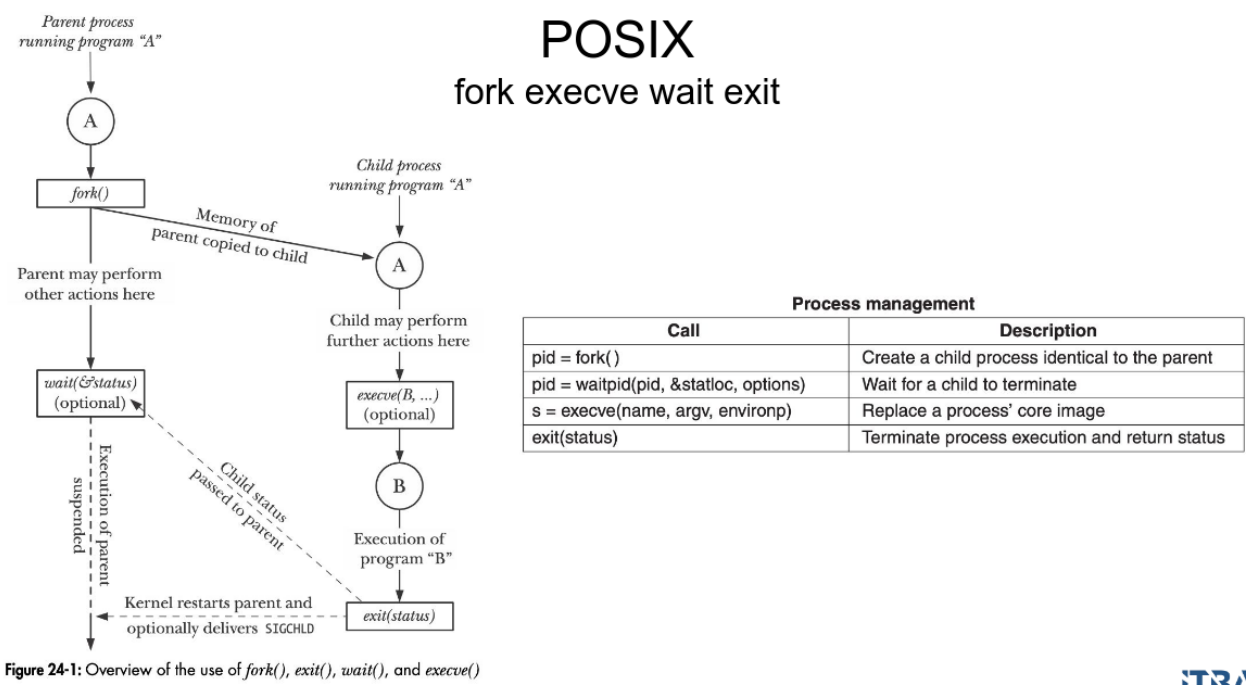

Figure 1: Pipeline fork

Como cada proceso puede crear varios procesos, pero tiene un solo padre, si quiero saber cual es el `pid` del

padre es necesario que el padre sepa cual es el `pid` del hijo. Esto se logra devolviendo el `pid` del hijo en el padre.

Es muy importante la llamada a `wait` porque sino queda un monton de procesos **undefined**, consume entradas en la tabla de procesos. Todas las variables que tenga un proceso original, incluyendo los punteros, el hijo los hereda. Es importante por ese motivo cerrar todos los **file descriptor**.

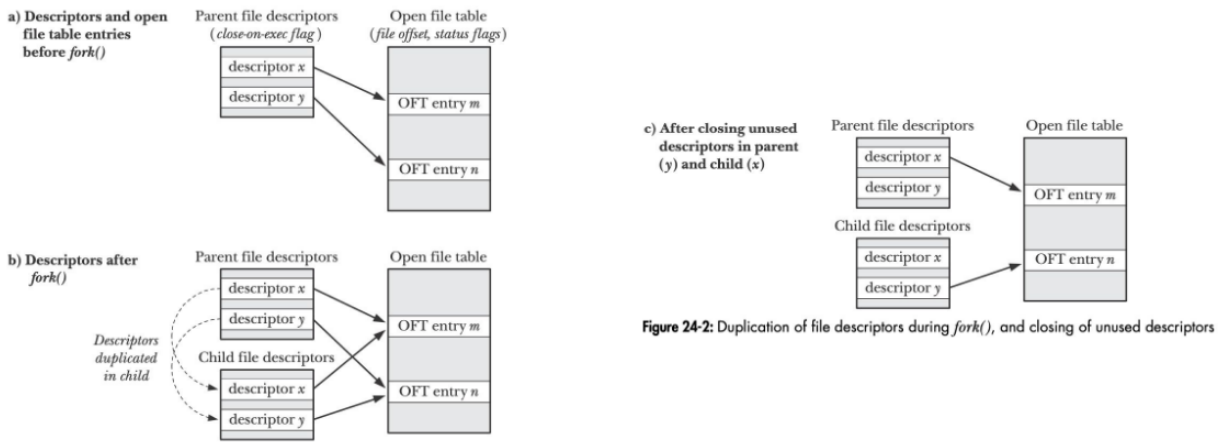

Figure 2: File descriptors fork

`ps`

Muy recomendado que lo aprendamos a usar bien para debuggear los TP. Se corre generalmente con `aux` pued indica: a+x -> mostrar todos los procesos, u -> incluir la columna con informacion del usuario de cada programa. Se puede combinar con otros arg(todo junto):

- Muestra los procesos que estan corriendo en el momento. Usando `w` como argumento(wide) muestra los argumentos que se utilizaron para correr cada programa.

- Si se corre con `e` va a mostrar las variables de entorno.
- Si se corre con `f` se mostrara un arbol de los procesos con caracteres ASCII

  Nota: Hay mas arg en la presentacion de campus.

Copy on write

Los procesos clones usan la misma memoria que el padre(heap, stack, memory maps), copiar esa memoria cada vez que usamos fork es lento y ocupa memoria. Entonces, linux permite que ambos procesos utilicen la misma memoria fisica y solo copia la memoria cuando alguno de los procesos intenta escribir. Linux implementa esta dandole la misma **page table** a cada programa, pero marcadas como **read only**. Entonces, cuando alguno intenta escribir ocurre una **page fault**.

File management

Todo lo que es I/O es universal en Unix, se maneja como si fuera un archivo. Es muy eficiente el read write, lo unico que cambia entre tipos como lo accedo. Para trabajar con archivos como humanos les asignamos

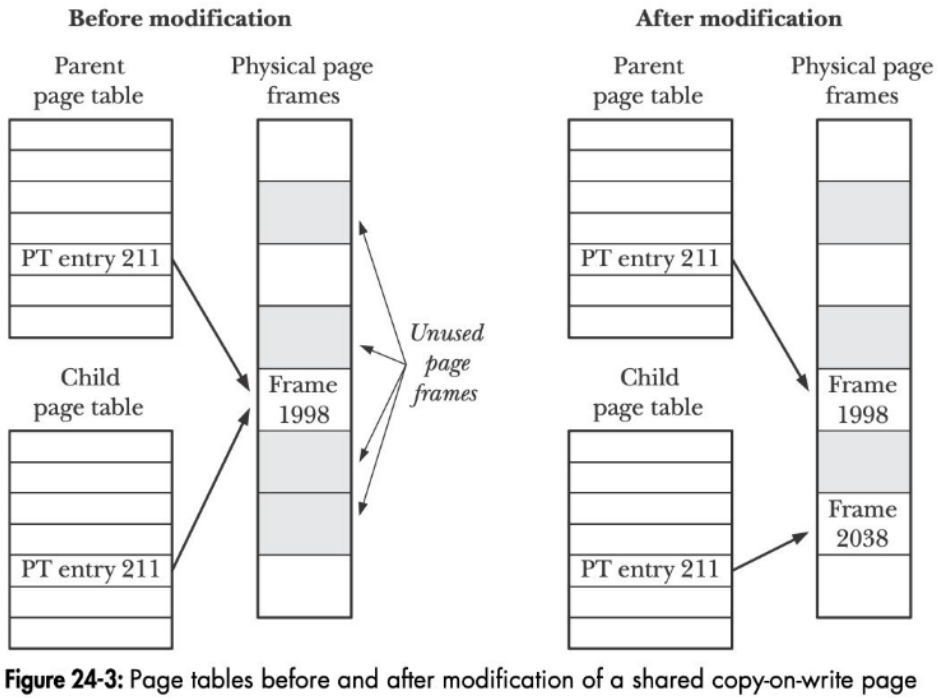

Figure 3: Representacion de copy on write

nombres, pero las computadoras prefieren tratar con numeros. Por lo tanto, cuando trabajamos con bajo nivel utilizamos **file descriptors** los cuales indican la referencia a un archivo **abierto**.

Una vez con el fd se puede leer y escribir, el fd 0 es la salida estandar. Comandos utiles:

- `head`, primeras lineas de un archivo
- `tail`, ultimas lineas de un archivo
- `stat`, muestra metadata del archivo
- `lseek`, corre el puntero a otra posicion

Una vez termino de trabajar con un fd, se libera y el numero queda disponible para cuando abra algun otro archivo. Cuando abro un archivo, usa el fd mas chico disponible. Cuando se lee de disco es muy importante que lo que uno lea o escriba este alineado con el **tamano del bloque** para evitar fragmentacion, no es obligatorio, pero es una buena practica.

Un archivo tiene un formato predefinido? No, para el sistema operativo no importa, hay metadata guardada en disco para saber su ubicacion o permisos, pero el archivo en si son bytes para el SO. Es problema de la aplicacion saber que formato tiene, si tubieran formatos para el SO seria demasiado complejo e innecesario.

> **Nota**: Cada proceso tiene su tabla con el mapeo de fd a nombre, es un error muy comun tener una tabla global para todos los procesos(esta mal eso). Todo lo que comentan es **local** al proceso. Si dos procesos tienen abierto el mismo archivo el fd no tienen porque coicidir.

`pipe`

Es un canal donde se envia todo lo que saca un proceso se lo manda a otro, es unidireccional y es un metodo de comunicacion. Es el mas sencillo que hay. Es un buffer compartido, donde un proceso escribe y otro lee, una cola. No hay posibilidad de reabrir un pipe, pues es un objeto anonimo(no tiene nombre).

En caso de que el programa que lee termine y el otro siga escribiendo se lanzara una senal de error nivel sistema operativo. No es comportamiento normal, por lo tanto se lanza un error. Detecta que el otro programa dejo de leer pues desaparece el fd. Pero que pasa si deja de escribir el programa, que pasara con el que lee? Se vacia el pipe y cuando en ultimo read devuelva 0, entonces se cierra el pipe.

Otro caso, el proceso que lee, lee despacio, entonces, se llena el buffer. Cuando la otra parte quiera escribir y el SO ve que el buffer esta lleno, bloquea la parte que escribe hasta que se pueda ejecutar write. No da error, pues esto es una situacion normal.

dup

El manejo del pipe se realiza de la siguiente manera:

1. Corro pipe en shell. Se llama syscall `pipe`
2. `pipe` devuelve a la shell fd escritura y fd lectura
3. Se hace un fork, de manera que los hijos(las dos shell) tienen los mismo fd que el padre(pues se hacen copias identicas).
4. La shell cierra los dos fd, pero siguen abiertos para los hijos. Se separa del pipe.
5. Se elimina el fd de lectura de un hijo y el de escritura para el otro. Quedando el diagrama de pipe convencional.
6. Luego para el proceso 1 se configura el stdout como el pipe y para el proceso 2 es el stdin. Duplicando los fd, cerrando stdout y stdin para los proceso.
7. Una vez duplicados a std y stdin, se cierran los originales.
8. Se hace el exec en las shell.

Aca child 1 y child 2 son una shell, al final del todo se hace el exec y se ejecutan los programas. Esta es una de las razones por la cual stderr esta separado de stdout.

> **Nota**: Para ver la representacion grafica ver apuntes. Puede haber dos fd, uno duplicado, abiertos, pero es mala practica.

Teorica 1: Introduccion

Sistemas operativos

Primero planteemos que ocurriria si no hubiese un sistema operativo, entonces, `printf` no funciona, no tiene sentido. No hay drivers, no hay nada definido y no existe el SO que puede proveer el servicio de imprimir en pantalla.

En conclusion, el sistema operativo provee servicios(resuelve problemas) a nivel de llamar una funcion que haga algo, abstrayendo los detalles(no me importa en que hardware lo este corriendo).

Provee un conjunto obstracto y limpio de los recursos, administra estos recursos. Se requiere que el hardware tenga por lo menos dos modos de operacion(con distintos privilegios) para que funcione correctamente.

Oculta el hardware y ofrece una interfaz limpia, elegante y consistente al programador, las abtracciones son una de las claves para comprender SOs.

Administrador de recursos

Que pasa si 2 o mas procesos intentan imprimir en la impresora?

El SO es un ente centralizado que agrega cada impresion a un queue y garantiza que el acceso al recurso impresora sea correcto.

Administrar recursos incluye **multiplexar** estos recursos, mediante alguno de los siguientes metodos:

- Tiempo
- Espacio

Multiplexar trata de la distribucion de un recurso(temporal o espacialmente), por ejemplo, dividir el auditorio de SDT en clases mas chicas(espacial).

Revision hardware

Procesador

Un registro importante es el RFLAGS.

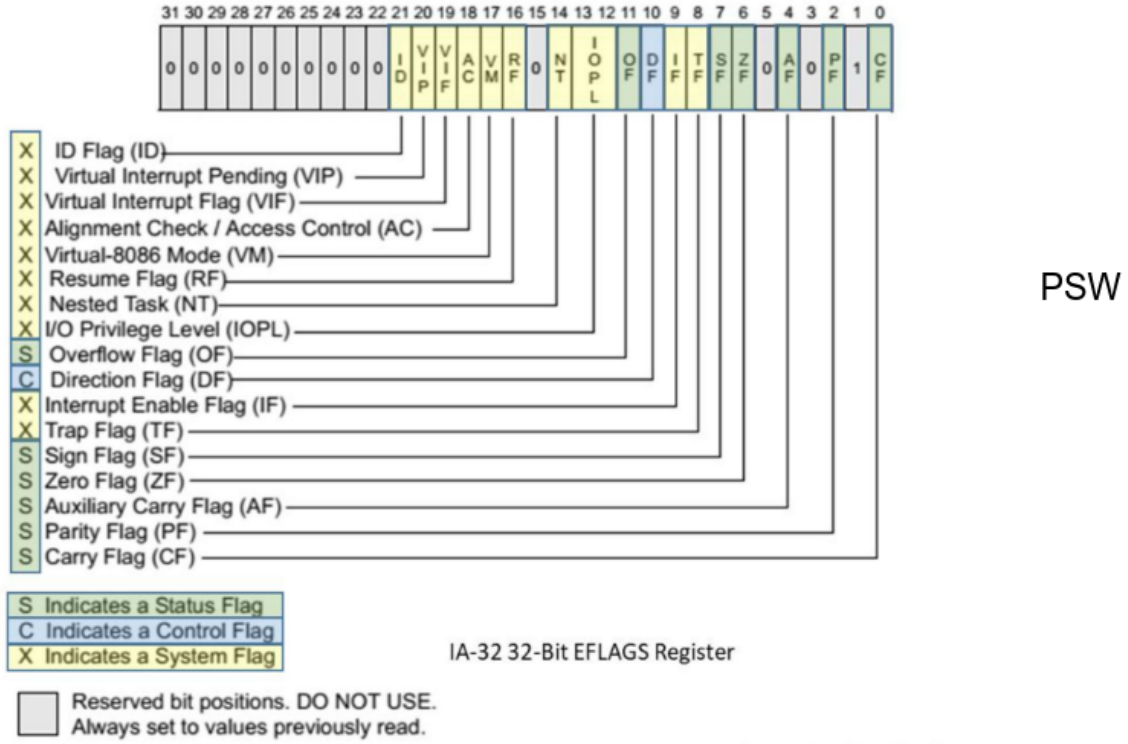

Figure 4: RFLAGS

Al iniciar el hardware, el stack estara vacio y el IP apuntara a la direccion de nuestro programa, pero primero hay que determinar cual es esa direccion.

Multithreading Mantener el estado de 2 threads e intercambiar entre ellos rápidamente cuando sea necesario. No es paralelismo.

Multicore Replicar los núcleos independientes. Pueden soportar múltiples threads.

> **Nota**: Ley de Moore es una aproximacion bastante acertada, pero llegamos a una limitacion de temperatura. Hoy en dia simplemente se replican los nucleos(muchos nucleos en un mismo procesador).

Memoria

Idealmente la memoria debería ser: - Extremadamente rápida (más que la ejecución de una instrucción) - Absurdamente grande (para albergar a todos los procesos juntos) - Asquerosamente barata

Obviamente esto es una utopia porque en la practica no se cumplen estas condiciones, por lo tanto, surgue una jerarquia de memoria(ver figura).

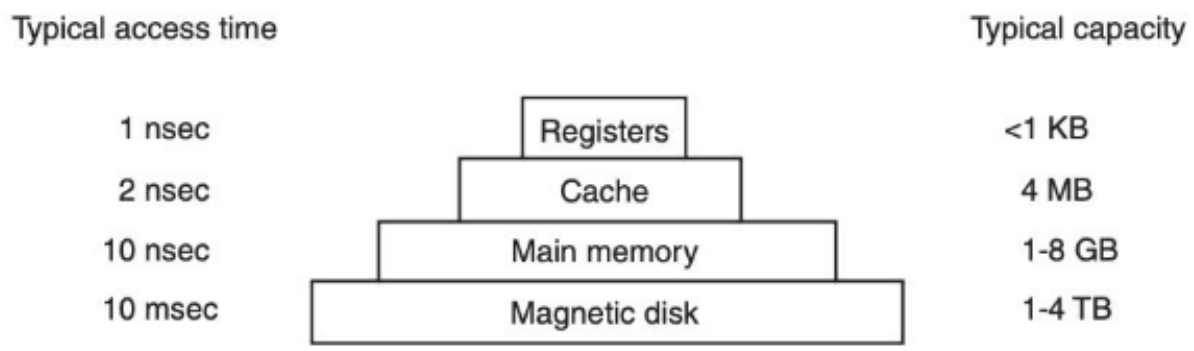

Figure 5: Jerarquia de los tipos de memoria

Dispositivos I/O

Un dispositivo consta de 2 partes:

- Controlador (abstracción)
- Dispositivo

Hay 3 formas de IO:

- Busy waiting(polling), en este caso, el cpu pregunta(es un derperdicio)
- Interrupción
- DMA (Direct Memory Access)

Proceso de interrupcion(?):

1. El driver le indica al controlador qué hacer. Este inicia la operación en el dispositivo.
2. El controlador señaliza al controlador de interrupciones (CI).
3. Cuando el CI está listo (otras interrupciones) setea el pin correspondiente en el CPU.
4. El CI escribe el número de dispositivo (vector de interrupciones) en el bus.

Booteo

- La placa madre posee un programa llamado BIOS (Basic I/O System)
- Flash RAM, no volátil pero actualizable por el SO
- La BIOS posee instrucciones para interactuar con la pantalla, teclado y disco

Proceso de booteo(una vez se determina cual es el dispositivo de booteo:

1. Chequear cuánta memoria tenemos y si ciertos dispositivos como el teclado están instalados y responden correctamente.
2. Escanea buses PCIe y PCI en busca de dispositivos instalados.
3. Determina el dispositivo de booteo.
4. Se carga el primer sector en memoria y se ejecuta
5. Este sector suele leer una tabla de particiones y se se carga un segundo sector
6. Se carga en memoria el SO y se ejecuta.
7. El SO consulta a la BIOS los dispositivos conectado

Tipos de SO

Ordenados de mas grandes a mas chicos.

- **Mainframe**: Mucho almacenamiento y memoria. Orientado a muchos procesos. Soporta trabajos por lotes, transacciones y timesharing. OS/390, tendencia a UNIX.
- **Server**: Computadoras personales o incluso mainframes. Impresoras, archivos o webs. Solaris, FreeBSD, Linux y Windows Server.
- **Multiproceso**: (Necesaria la separación)
- **Personal Computer**: Proveer soporte a un único usuario. Linux, FreeBSD, Windows y Apple's OS X.
- **HandHeld Computer**: Tablets y smartphones. Android y iOS
- **Embedded**: Electrodomésticos y teléfonos antiguos. ROM, no permite la instalación de aplicaciones. Embedded Linux, QNX y VxWorks.
- **Sensor-node**: Redes de nodos. Poseen CPU RAM y ROM. Sensan diferentes condiciones. Conexión inalámbrica. TinyOS
- **Real Time**: Necesitamos que una acción ocurra en un momento específico. hard vs soft. Industria, aviónica, milicia y multimedia. eCos y FreeRTOS
- **Smart Card**: Pagos electrónicos. Del tamaño de una tarjeta de crédito. Restricciones de potencia y memoria muy altas. Obtienen energía por inducción o al conectarse a un lector.

System calls

Como vimos en arqui, las syscall buscan:

- Proveer un conjunto abstracto y limpio de los recursos (hardware) -> entender qué hace el SO
- Administrar estos recursos -> transparente para el proceso
- Se puede ver como una simple llamada a función que además cambia a modo kernel
- El mecanismo para realizar una system call depende de la arquitectura y debe expresarse en assembler.

POSIX es basicamente la API de linux, en windows se llama **win32**, nosotros vamos a ver el grupo principal de syscalls definidos por POSIX.

> **Nota**: Malloc puede derivar de una syscall, pero en realidad solo es necesario realizar una syscall cuando se acaba la memoria disponible para pedirle mas memoria al kernel.

Read

Para verificar que la primera vez que se ejecuto `read` levanto todos los datos posibles, se corre una segunda vez, si devuelve 0 significa que no hay mas nada para leer (si le pedis 1024 y te da 1000 y despues 0, entonces el archivo es de 1000 bytes) y la lectura fue exitosa. Sin embargo, si la segunda vez devuelve mas que 0(en el caso de leer un archivo) entonces no se leyo todo y hay que volver a hacerlo.

Waitpid

Hace que el proceso espere a que termine el proceso con el pid especificado. Durante la espera el programa se queda bloqueado y no ocupa ciclos del procesador, pero si memoria. Se entera el parent porque el SO termina la syscall.

Por otro lado, el SO se entera de que termino por la llamada a `exit()`(la cual es una syscall) del child. Se fija quien es el parent y si el mismo esta esperando lo notifica.

> **Nota**: Hay mas syscall en la T1.

Kill

Manda una aenal que alerta a otro proceso, simplemente le avisa, pero no necesariamiente lo mata. Ahora, si envio la senal -9 ahi si termina. Cada programa tiene un handler para las senales, entonces los handlers se general en runtime y se setean con syscalls. Esto permite que un programa en vez de terminar muestre una ventana que pregunte al usuario si esta seguro de que quiere terminar el proceso.

El SO hara que el IP cambie a apuntar la funcion del handler, lo hara sin importar si la funcion no esta en su lugar y la va a ir a buscar. Implementa **zero trust**, se va a asegurar que el programa termine si se envia la senal de terminacion.

Fork

Mis palabras:

> Se crea un proceso identico que sigue la ejecucion en el momento que retorna.

Asi es, **KISS**. Es un ejemplo de las varias funciones no siguen lo usual(exec(no retorna), exit(no retorna), etc.): llamo una vez y retornan una vez.

A donde va la salida estandar

Ejecutan write en fd 1, no les importa a donde apunta el fd, eso depende del SO. Algunos ejemplos:

- `ls` -> terminal,
- `ls | grep` -> pipe
- `ls > foo.txt` -> archivo
- `ls > /dev/null` -> dispositivo

Estructura de un sistema operativo

Monolithic systems

Toda la funcionalidad del SO esta en un solo binario, toda funcion tiene visibilidad del resto de las funciones. Esto es eficiente, pero complejo. Un error en cualquier funcion hace fallar el sistema operativo completo. Ademas, soporta **shared libraries** o **DDLs**.

Presenta una filosofia de estructura, ver figura. El TP arqui es monolitico(como el TPE de SO), el user space esta separado, pero no es parte del SO.

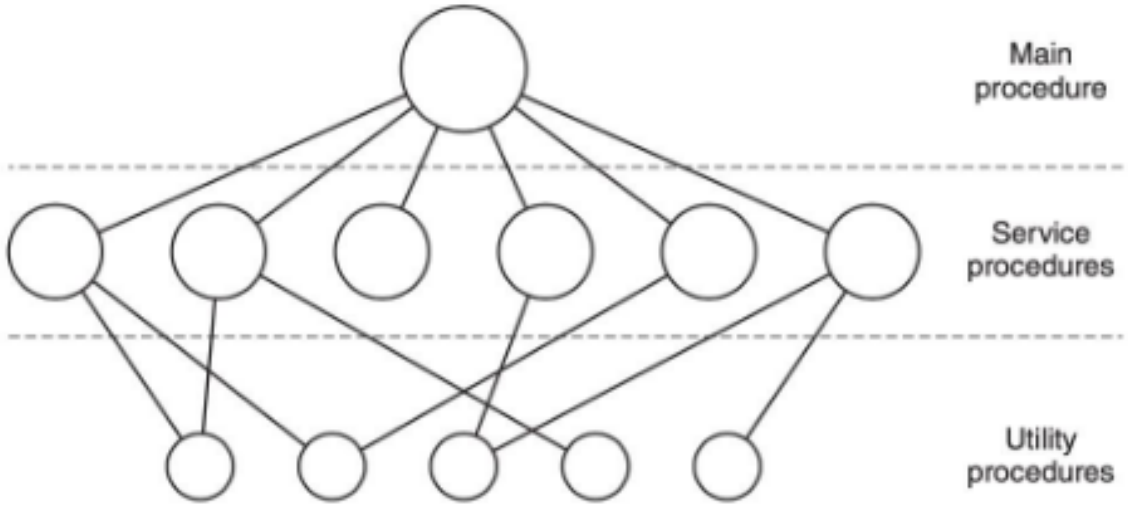

Figure 6: SO monolitico

Nota: Es muy usado, por ejemplo, Windoes, FreeBSD, Linux, etc.

Microkernels

La idea es dejar los componentes esenciales en el kernel(lo minimo), mientras que el resto se va a correr como clientes del kernel(en user-mode). Los drivers corren como un proceso mas, por lo que un error fuera del kernel no hace fallar al SO. Tiene alta confiabilidad(Real time), implementado por Integrity, PikeOS, MINIX, etc.

Un ejemplo, MINIX, presenta un **reincarnation server** el cual es el padre de todos los drivers y servers, chequea constantemente de controles que esten todos vivos. Limpia los drivers y servers muertos, y chequea una tabla para ver que hacer(generalmente los reinicia).

Mecanismo vs politica La separacion entre mecanismo y politica es una buena estrategia de programacion pues permite reducir el tamano del kernel y separar areas diferentes. Colocar el mecanismo en el kernel y la política por fuera, por ejemplo, asignar prioridades a los procesos(por fuera del kernel) y ejecutarlos según estas prioridades:

- Elegir al proceso de mayor prioridad (mecanismo) puede estar en el kernel

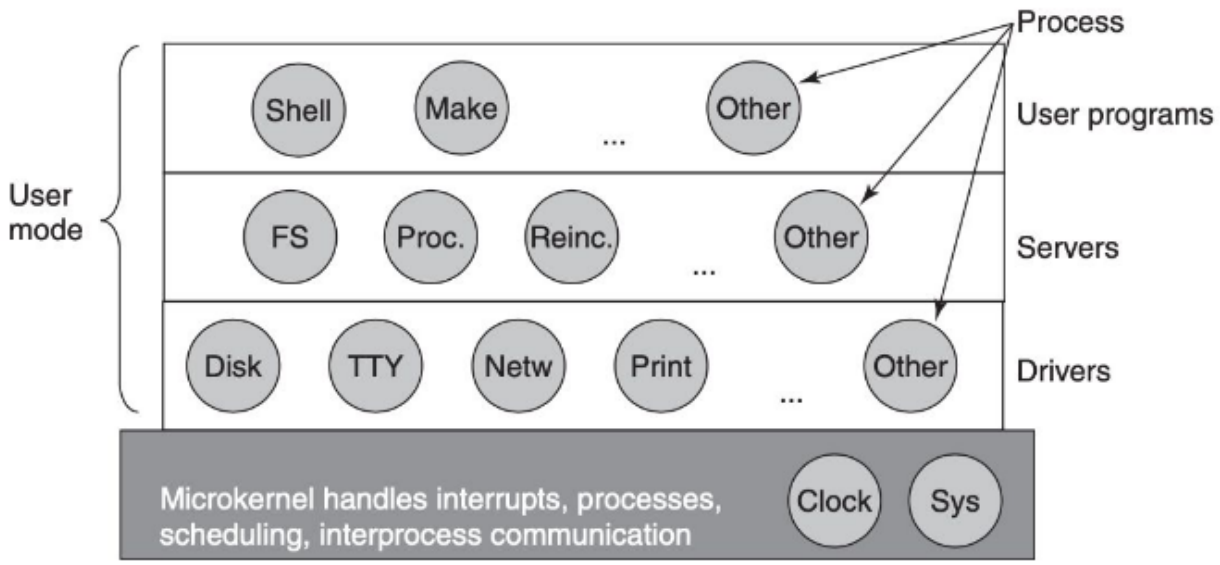

Figure 7: SO microkernels

- Asignar las prioridades (política) puede estar por fuera del kernel.

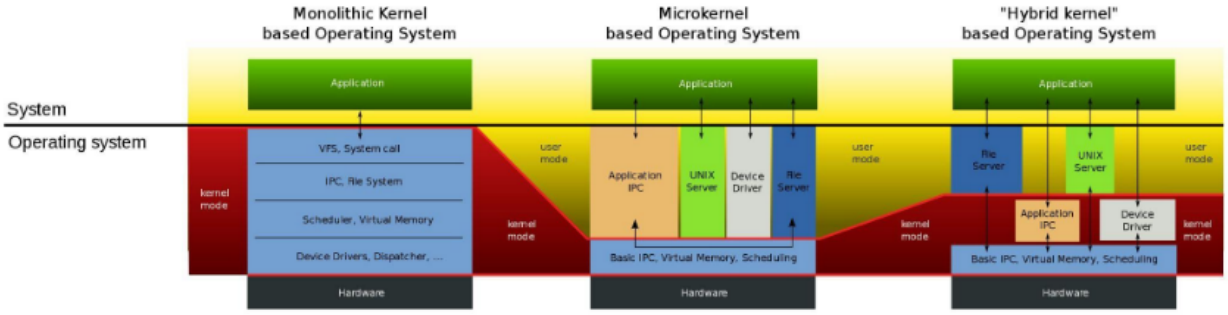

Figure 8: Monolitico vs Microkernels

Nota: Hay implementaciones hibridas que combinan ambas.

Virtual machines

Tradicionalmente Mail server, FTP server, Web server, etc corren en computadoras separadas, lo cual lo hace popular para Web hosting: servidor dedicado vs. compartido - costo y flexibilidad Tambien presentan utilidad para uso personal. Para que funcione, el CPU debe ser virtualizable: SO corriendo en user mode ejecuta instrucción privilegiada, es necesario que haga un TRAP al virtualizador para emularla en software. Pentium ignoraba estas instrucciones aunque existían intérpretes como Bochs (bajo rendimiento).

No es exactamente lo mismo que un emulador de hardware, pero si hace creer a los SO dentro de los VM que son su propia maquina con su propio CPU, RAM, etc. Son maquinas separadas, lo cual lo hace conveniente por cuestiones de costo y seguridad.

Hay diferentes tipos de maquinas virtuales, tambien conocidos como **hypervisors**:

- **Virtual machine monitor o type 1 hypervisor**: Se ocupa de proveer los servicios necesarios.
- **Type 2 hypervisor**: Tiene un SO por debajo que se encarga de abstraer los procesos, file system, etc. QEMU y BOCHS.
- Sistemas híbridos.

Es mas eficiente y rapido el tipo 1 debido a que tiene menos capas e interactua directamente con el SO. El tipo 2 tiene ya una capa extra.

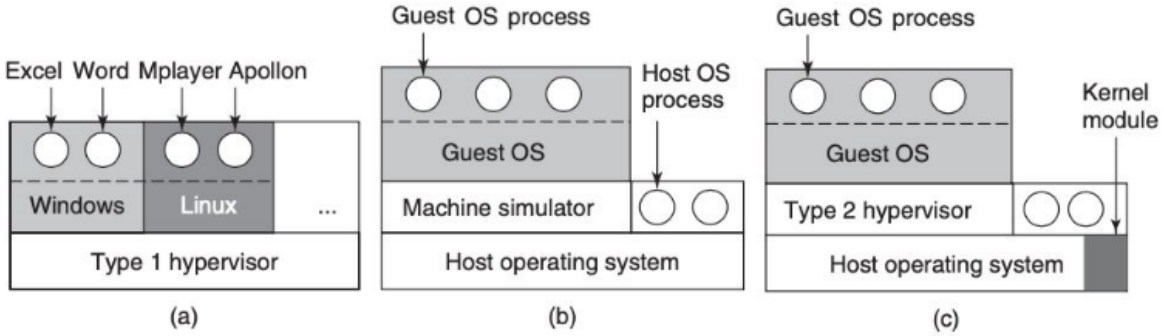

Figure 9: Tipos de VMs